

DEDICATED TO INNOVATION

www.bacancy.com





**Customer Satisfaction
Is Our Highest Priority**



Employee Matters



In this session

What we are learning in this session

- ▶ Introduction
- ▶ Create ASP.NET Core Web API

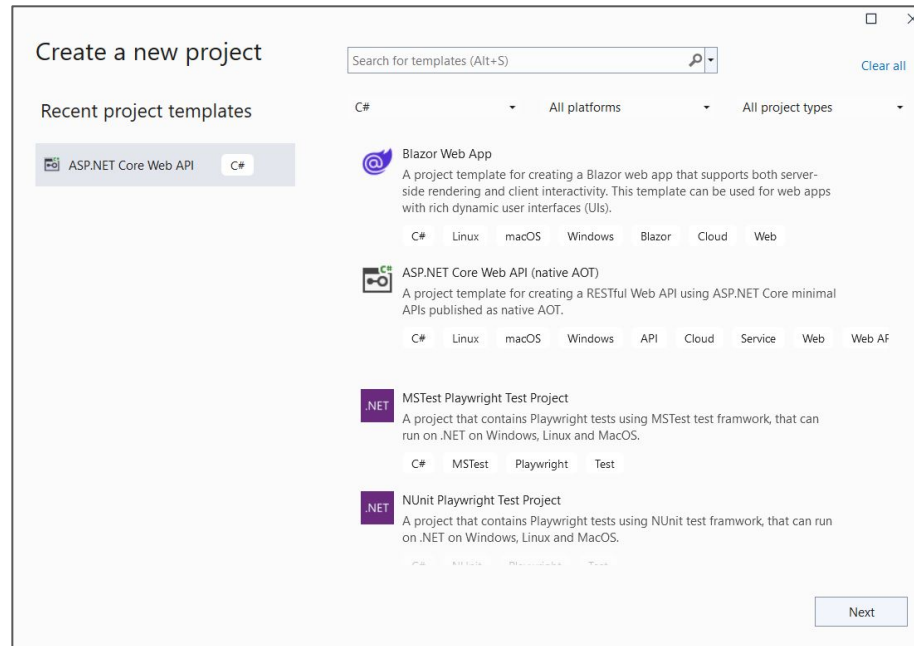


Introduction

- ▶ Authentication is the process of validating user credentials, and authorization is the process of checking privileges for a user to access specific modules in an application. In this article, we will see how to protect an ASP.NET Core Web API application by implementing JWT authentication.
- ▶ We will also see how to use authorization in ASP.NET Core to supply access to various functionalities of the application. We will store user credentials in an SQL server database, and we will use Entity Framework and Identity framework for database operations.
- ▶ JSON Web Token (JWT) is an open standard (RFC 7519) that defines a compact and self-contained way for securely transmitting information between parties as a JSON object. This information can be verified and trusted because it is digitally signed. JWTs can be signed using a secret (with the **HMAC** algorithm) or a public/private key pair using **RSA** or **ECDSA**.
- ▶ In its compact form, JSON Web Tokens consist of three parts separated by dots (.), which are:
 - Header
 - Payload
 - Signature
- ▶ Therefore, a JWT typically looks: **xxxx.yyyy.zzzz**

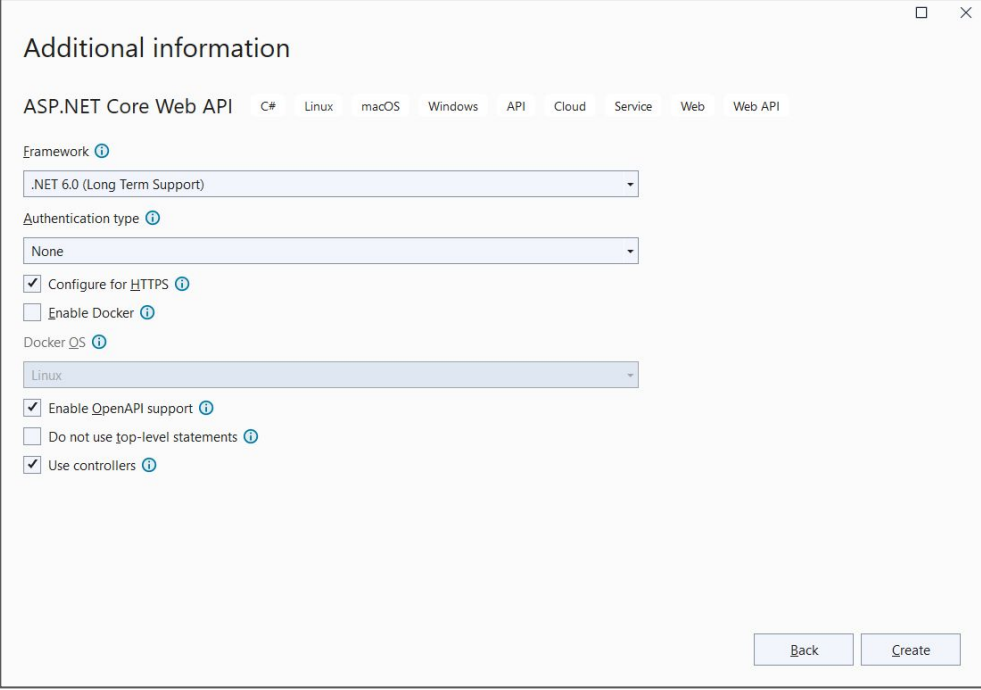
Create ASP.NET Core Web API

- ▶ We need Visual Studio 2022 to create .NET 6.0 applications. We can choose the ASP.NET Core Web API template from Visual Studio 2022.



Create ASP.NET Core Web API

- ▶ We can give a suitable name for our project and choose the .NET 6.0 framework.



The screenshot shows the 'Additional information' dialog box in Visual Studio. The title bar includes a maximize button and a close button. The dialog has a tabbed interface with the following tabs: ASP.NET Core Web API (selected), C#, Linux, macOS, Windows, API, Cloud, Service, Web, and Web API. The 'Framework' dropdown is set to '.NET 6.0 (Long Term Support)'. The 'Authentication type' dropdown is set to 'None'. There are three checkboxes: 'Configure for HTTPS' (checked), 'Enable Docker' (unchecked), and 'Docker OS' (set to 'Linux'). There are also three checkboxes at the bottom: 'Enable OpenAPI support' (checked), 'Do not use top-level statements' (unchecked), and 'Use controllers' (checked). At the bottom right, there are 'Back' and 'Create' buttons.

Additional information

ASP.NET Core Web API C# Linux macOS Windows API Cloud Service Web Web API

Framework ⓘ
.NET 6.0 (Long Term Support)

Authentication type ⓘ
None

☒ Configure for HTTPS ⓘ
☐ Enable Docker ⓘ

Docker OS ⓘ
Linux

☒ Enable OpenAPI support ⓘ
☐ Do not use top-level statements ⓘ
☒ Use controllers ⓘ

Back Create

Create ASP.NET Core Web API

- ▶ Our new project will be created in a few moments.
- ▶ We must install the 4 libraries below into the new project.
 - Microsoft.EntityFrameworkCore.SqlServer
 - Microsoft.EntityFrameworkCore.Tools
 - Microsoft.AspNetCore.Identity.EntityFrameworkCore
 - Microsoft.AspNetCore.Authentication.JwtBearer
- ▶ You can use the NuGet package manager to install the above packages.
- ▶ We can change the appsettings.json with values below. It has database connection details and other details for JWT authentication.

Create ASP.NET Core Web API

► appsettings.json

```
1  {
2    "Logging": {
3      "LogLevel": {
4        "Default": "Information",
5        "Microsoft.AspNetCore": "Warning"
6      }
7    },
8    "AllowedHosts": "*",
9    "ConnectionStrings": {
10     "ConnStr": "Data Source=(localdb)\\MSSQLLocalDB;Initial Catalog=JWTAuthDB;Integrated
11               Security=True;ApplicationIntent=ReadWrite;MultiSubnetFailover=False"
12   },
13   "JWT": {
14     "ValidAudience": "http://localhost:4200",
15     "ValidIssuer": "http://localhost:5000",
16     "Secret": "JWTAuthenticationHIGHsecuredPasswordVVVp10H7Xzyr"
17   }
18 }
```

Create ASP.NET Core Web API

- ▶ We can create a new folder, “Auth” and create the “ApplicationDbContext” class under Auth folder and add below code. We will add all the classes related to authentication under the Auth folder.
- ▶ **ApplicationDbContext.cs**

```
1  using Microsoft.AspNetCore.Identity;
2  using Microsoft.AspNetCore.Identity.EntityFrameworkCore;
3  using Microsoft.EntityFrameworkCore;
4
5  namespace JWTAuthentication.NET6._0.Auth
6  {
7      6 references
8      public class ApplicationDbContext : IdentityDbContext<IdentityUser>
9      {
10         0 references
11         public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options) : base(options)
12         {
13             0 references
14             protected override void OnModelCreating(ModelBuilder builder)
15             {
16                 base.OnModelCreating(builder);
17             }
18         }
19     }
```

Create ASP.NET Core Web API

- ▶ Create a static class “UserRoles” and add below values.
- ▶ UserRoles.cs

```
1 namespace JWTAuthentication.NET6._0.Auth
2 {
3     9 references
4     public static class UserRoles
5     {
6         public const string Admin = "Admin";
7         public const string User = "User";
8     }
```

- ▶ We have added two constant values “Admin” and “User” as roles. You can add as many roles as you wish.

Create ASP.NET Core Web API

- ▶ Create class "RegisterModel" for new user registration.
- ▶ RegisterModel.cs

```
1  using System.ComponentModel.DataAnnotations;
2
3  namespace JWTAuthentication.NET6._0.Auth
4  {
5      2 references
6      public class RegisterModel
7      {
8          [Required(ErrorMessage = "User Name is required")]
9          4 references
10         public string? Username { get; set; }
11
12         [EmailAddress]
13         [Required(ErrorMessage = "Email is required")]
14         2 references
15         public string? Email { get; set; }
16
17         [Required(ErrorMessage = "Password is required")]
18         2 references
19         public string? Password { get; set; }
20     }
21 }
```

Create ASP.NET Core Web API

- ▶ Create class "LoginModel" for user login.
- ▶ LoginModel.cs

```
1  using System.ComponentModel.DataAnnotations;
2
3  namespace JWTAuthentication.NET6._0.Auth
4  {
5      1 reference
6      public class LoginModel
7      {
8          [Required(ErrorMessage = "User Name is required")]
9          1 reference
10         public string? Username { get; set; }
11
12         [Required(ErrorMessage = "Password is required")]
13         1 reference
14         public string? Password { get; set; }
15     }
16 }
```

Create ASP.NET Core Web API

- ▶ We can create a class “Response” for returning the response value after user registration and user login. It will also return error messages if the request fails.

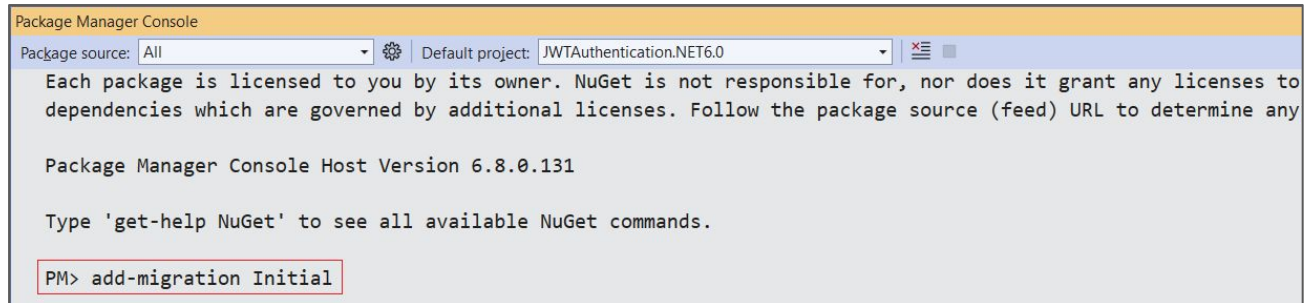
▶ Response.cs

```
1 namespace JWTAuthentication.NET6._0.Auth
2 {
3     6 references
4     public class Response
5     {
6         6 references
7         public string? Status { get; set; }
8         6 references
9         public string? Message { get; set; }
10    }
11 }
```

- ▶ We can create an API controller “AuthenticateController” inside the “Controllers” folder. Kindly check mentioned controller in Source Code.

Create ASP.NET Core Web API

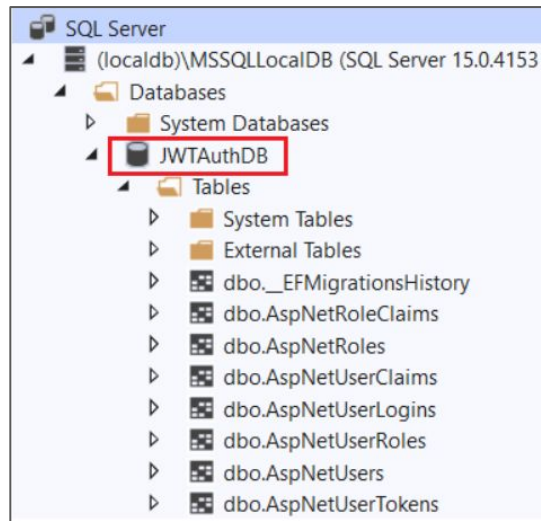
- ▶ We have added three methods “login”, “register”, and “register-admin” inside the controller class. Register and register-admin are almost same, but the register-admin method will be used to create a user with admin role. In login method, we have returned a JWT token after successful login.
- ▶ In .NET 6.0, Microsoft removed the Startup class and only kept Program class. We must define all our dependency injection and other configurations inside the Program class. Kindly check mentioned file in Source Code.
- ▶ We must create a database and tables needed before running the application. As we are using entity framework, we can use below database migration command with package manger console to create a migration script.
- ▶ **add-migration Initial**



```
Package Manager Console
Package source: All Default project: JWTAuthentication.NET6.0
Each package is licensed to you by its owner. NuGet is not responsible for, nor does it grant any licenses to dependencies which are governed by additional licenses. Follow the package source (feed) URL to determine any
Package Manager Console Host Version 6.8.0.131
Type 'get-help NuGet' to see all available NuGet commands.
PM> add-migration Initial
```

Create ASP.NET Core Web API

- ▶ Use the command below to create database and tables.
- ▶ **update-database**
- ▶ If you check the database using SQL server object explorer, you can see that the tables below are created inside the database.
- ▶ There are 7 tables created for User, Role and Claims during the database migration process. This is due to the Microsoft Identity framework.
- ▶ ASP.NET Core Identity is a membership system which allows you to add login functionality to your application. Users can create an account and login with a username and password, or they can use an external login provider such as Facebook, Google, Microsoft Account, Twitter and more.
- ▶ You can configure ASP.NET Core Identity to use a SQL Server database to store usernames, passwords, and profile data.



Create ASP.NET Core Web API

- ▶ We can add “Authorize” attribute inside the “WeatherForecast” controller.

```
[Authorize]
[ApiController]
[Route("api/[controller]")]
3 references
public class WeatherForecastController : ControllerBase
{
    private static readonly string[] Summaries = new[]
    {
        "Freezing", "Bracing", "Chilly", "Cool", "Mild", "Warm", "Balmy", "Hot", "Sweltering", "Scorching"
    };
};
```

- ▶ We can add “Authorize” attribute inside the “WeatherForecast” controller.

GET

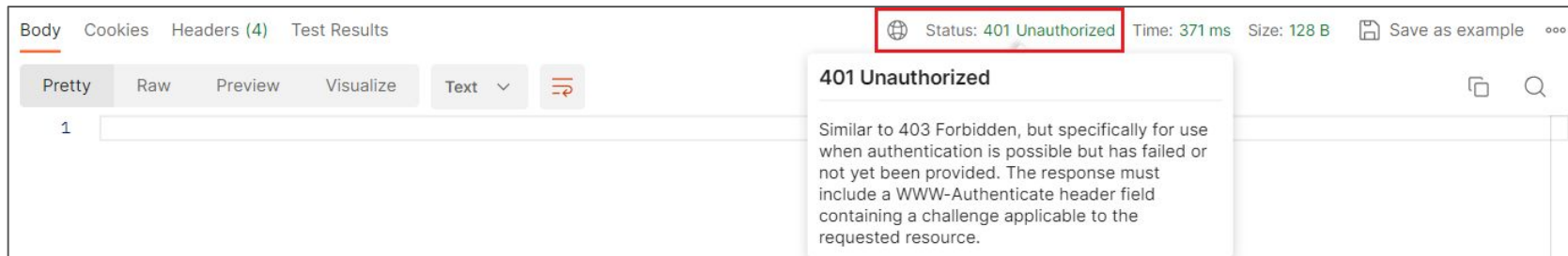
Params Authorization Headers (7) Body Pre-request Script Tests Settings

Query Params

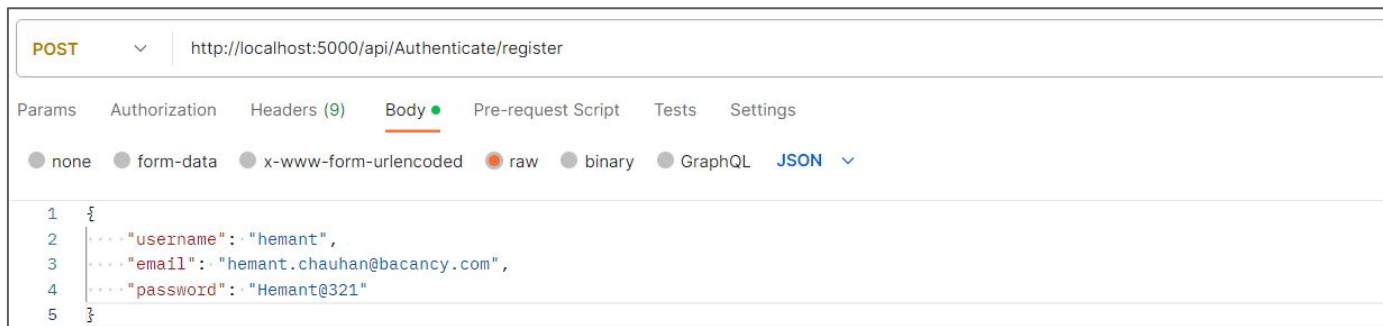
	Key	Value	Description
	Key	Value	Description

Create ASP.NET Core Web API

- ▶ We have received a 401 unauthorized error. Because, we have added Authorize attribute to entire controller. We must supply a valid token via request header to access this controller and methods inside the controller.



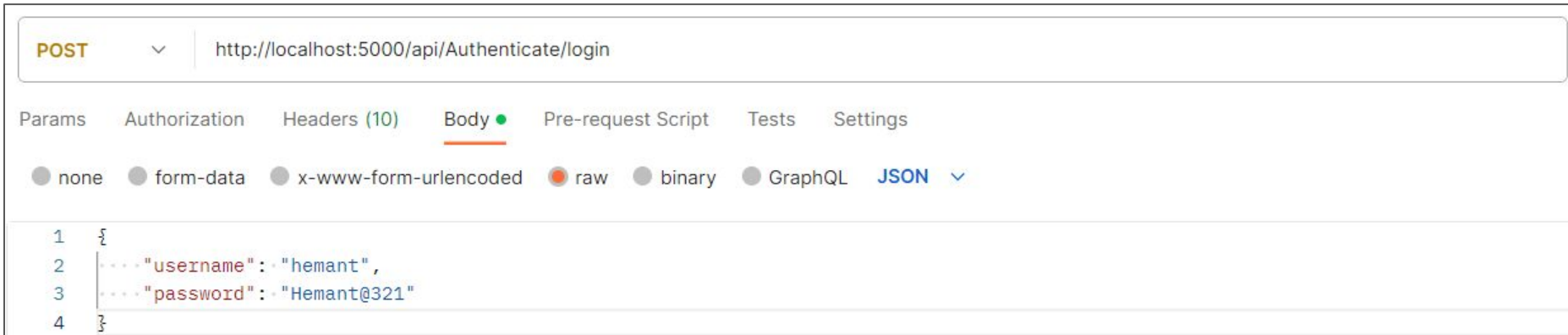
- ▶ We can create a new user using a register method in authenticate controller.



Create ASP.NET Core Web API



- ▶ We have supplied the input data as raw JSON format.
- ▶ We can use the above user credentials to login and get a valid JWT token.



Create ASP.NET Core Web API



The screenshot shows a web browser's developer tools interface. The 'Body' tab is selected, displaying a JSON response. The status bar at the top indicates 'Status: 200 OK', 'Time: 1021 ms', and 'Size: 535 B'. The JSON response is formatted as follows:

```
1 {  
2   "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.  
   eyJodHRwOi8vc2NoZW1hcy54bWxzbnB2FwLm9yZy93cy8yMDA1LzA1L2lkZW50aXR5L2NsYWltcy9uYW1lIjoiaGVtYW50IiwianRpIjoiaWNTK1OWRlYjgtNWl5ZS00MmVjLWlWn  
   TgtZDVkNDVmYTE4M2Q4IiwiaXhwIjoxNzAyNDcyOTE5LCJpc3MiOiJodHRwOi8vbG9jYXRob3N0OjUwMDA1LCJhdWQiOiJodHRwOi8vbG9jYXRob3N0OjYMDAifQ.  
   r_DZ9q-dHKXMBM2M-Q4bdX4da2tc1V0o61bUDxikJs0",  
3   "expiration": "2023-12-13T13:08:39Z"  
4 }
```

- ▶ We have received a token after successful login with the above credentials.
- ▶ We can decode the token and see the claims and other information using <https://jwt.io> site.

Create ASP.NET Core Web API



Libraries

Ask

Auth0
by Okta

②

PASTE A TOKEN HERE

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJodHRwOi8vc2NoZW1hcy54bWxzY2FwLm9yZy93cy8yMDA1LzA1L2lkZW50aXR5L2NsYWltcy9uYW11IjoiaGVtYW50IiwianRpIjoiaWNTk1OWRlYjgtNWl5ZS00MmVjLWlIwNTgtZDVkNDVmYTE4M2Q4IiwiaXhwIjoxNzAyNDcyOTE5LCJpc3MiOiJodHRwOi8vbG9jYWxob3N0OjUwMDAiLCJhdWQiOiJodHRwOi8vbG9jYWxob3N0OjQyMDAifQ.r_DZ9q-dHKXMBM2M-Q4bdX4da2tc1V0o61bUDxikJs0

EDIT THE PAYLOAD AND SECRET

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

```
{
  "http://schemas.xmlsoap.org/ws/2005/05/identity/claims/
name": "hemant",
  "jti": "5959deb8-5b9e-41ec-b058-d5d45fa183d8",
  "exp": 1702472919,
  "iss": "http://localhost:5000",
  "aud": "http://localhost:4200"
}
```

```
HMACSHA256(  
    base64UrlEncode(header) + ".",  
    base64UrlEncode(payload),  
    your-256-bit-secret  
)
```

Create ASP.NET Core Web API

- ▶ We can pass the above token value as a **Bearer** token inside the authorization tab and call get method of weatherforecast controller again.

The screenshot shows the Postman interface for a GET request to `http://localhost:5000/api/WeatherForecast`. The **Authorization** tab is selected, showing a **Bearer Token** type. The token value is `eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ...`. A warning message states: "Heads up! These parameters hold sensitive data. To keep this data secure while working in a collaborative environment, we recommend using variables. Learn more about [variables](#)".

The **Body** tab is selected, showing the response status **Status: 200 OK**, time `45 ms`, and size `647 B`. The response is displayed in **JSON** format:

```
1 [
2   {
3     "date": "2023-12-14T15:48:19.5440854+05:30",
4     "temperatureC": -18,
5     "temperatureF": 0,
6     "summary": "Chilly"
7   },
8 ]
```

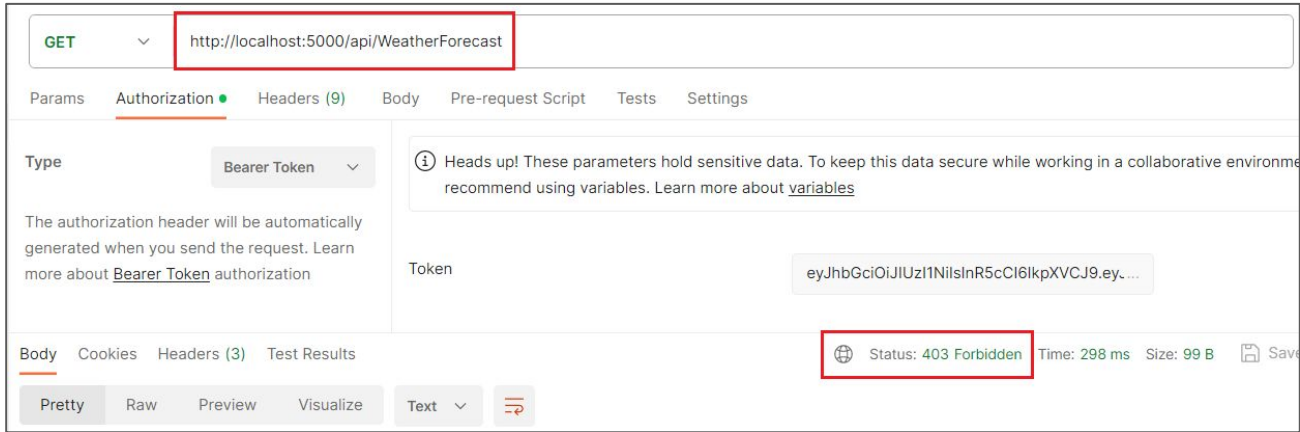
Create ASP.NET Core Web API

- ▶ This time, we have successfully received the values from the controller.
- ▶ We can change the weatherforecast controller with role-based authorization.

```
[Authorize(Roles = UserRoles.Admin)]  
[ApiController]  
[Route("api/[controller]")]  
3 references  
public class WeatherForecastController : ControllerBase  
{  
    private static readonly string[] Summaries = new[]  
    {  
        "Freezing", "Bracing", "Chilly", "Cool", "Mild", "Warm", "Balmy", "Hot", "Sweltering", "Scorching"  
    };  
};
```

- ▶ Now, only users with admin role can access this controller and methods.
- ▶ We can try to access the weatherforecast controller with same token again in Postman tool.

Create ASP.NET Core Web API



- ▶ We have received a 403 forbidden error instead of 401 now. Even though we are passing a valid token we don't have sufficient privilege to access the controller. To access this controller, the user must have an admin role permission. Current user is a normal user and does not have any admin role permission.
- ▶ We can create a new user with an admin role permission. We already have a method "register-admin" in authenticate controller for the same purpose.

Create ASP.NET Core Web API

The screenshot displays a REST client interface with the following details:

- Method:** POST
- URL:** `http://localhost:5000/api/Authenticate/register-admin` (highlighted with a red box)
- Body:** The request body is in JSON format, containing:

```
1 {  
2   "username": "hemant-admin",  
3   "email": "hemant.chauhan@bacancy.com",  
4   "password": "Hemant@321"  
5 }
```
- Response:** The response is also in JSON format, showing:

```
1 {  
2   "status": "Success",  
3   "message": "User created successfully!"  
4 }
```
- Status:** 200 OK (highlighted with a red box)
- Time:** 847 ms
- Size:** 207 B
- Buttons:** Pretty, Raw, Preview, Visualize, JSON, and Save.

- We can use this token instead of the old token to access the weatherforecast controller.

Create ASP.NET Core Web API

GET http://localhost:5000/api/WeatherForecast

Params **Authorization** Headers (9) Body Pre-request Script Tests Settings

Type **Bearer Token**

The authorization header will be automatically generated when you send the request. Learn more about [Bearer Token](#) authorization

Token: eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ...

Heads up! These parameters hold sensitive data. To keep this data secure while working in a collaborative environment, we recommend using variables. Learn more about [variables](#)

Body Cookies Headers (4) Test Results **Status: 200 OK** Time: 34 ms Size: 648 B Save

Pretty Raw Preview Visualize JSON

```
1 [
2   {
3     "date": "2023-12-14T17:28:55.7140675+05:30",
4     "temperatureC": 32,
5     "temperatureF": 89,
6     "summary": "Balmy"
7   },
```

► Now we have successfully fetched the data from weatherforecast controller.

Thank you